

用OpenMP并行实现CYK算法

CYK算法（Cocke–Younger–Kasami algorithm，缩写为CYK）是由约翰·科克，Younger和嵩忠雄共同研究出来，大约发表于1965年的一个用来判定任意给定的字符串w是否属于一个上下文无关文法的编译算法。

普通的回溯法在最坏的情况下需要指数时间才能解决这样的问题，而CYK算法只需要多项式时间就够了($O(n^3)$ ，n为字符串w的长度)。CYK算法采用了动态规划的思想。伪代码描述如下：

```
function CKY-PARSE(words, grammar) returns table

for j ← from 1 to LENGTH(words) do
    table[j - 1, j] ← {A | A → words[j] ∈ grammar }
    for i ← from j - 2 downto 0 do
        for k ← i + 1 to j - 1 do
            table[i, j] ← table[i, j] ∪
                {A | A → BC ∈ grammar,
                  B ∈ table[i, k],
                  C ∈ table[k, j] }
```

本作业提供了串行代码与数据样例，请同学们在串行代码的基础上完成代码并行化。串行代码仅供参考，也可以自己编写效率更高的串行代码。

不要尝试修改算法，因为编译原理课上讲的算法不能处理二义性文法，测试用例给定的文法都是有二义性的上下文无关文法。

也不可对文法进行化简，这会导致结果不正确。

【输入形式】

从“input.txt”读入。

第一行是一个整型，表示非终结符的数量vn_num；

第二行是一个整型，表示“右侧是两个非终结符的产生式”的数量production2_num，即代码中的Production2类型的数量；

接下来production2_num行，每行是一个Production2类型，非终结符的格式是“<%d>”，数字范围是0到(production2_num-1)，“<0>”表示开始符号；

然后下一行是一个整型，表示“右侧是一个终结符的产生式”的数量production1_num，即代码中的Production1类型的数量；

接下来production1_num行，每行是一个Production1类型，终结符的格式是“%c”，范围是所有可打印的ASCII字符；

然后下一行是一个整型，表示被测试的字符串的长度；

最后一行是被测试的字符串。

【输出形式】

一个32位无符号整型，表示被测试的字符串能构成多少棵符合文法的语法树，语法树的数量可能很多。为了简单起见，若发生溢出就截取低32位。

【样例输入】

```
4
5
<0>::=<1><2>
<0>::=<2><3>
<1>::=<2><1>
<2>::=<3><3>
<3>::=<1><2>
3
<1>::=a
<2>::=b
<3>::=a
5
baaba
```

【样例输出】

```
2
```

【样例说明】

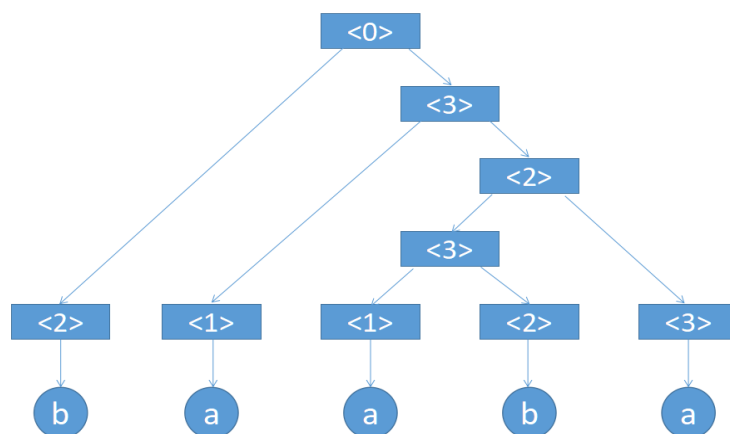
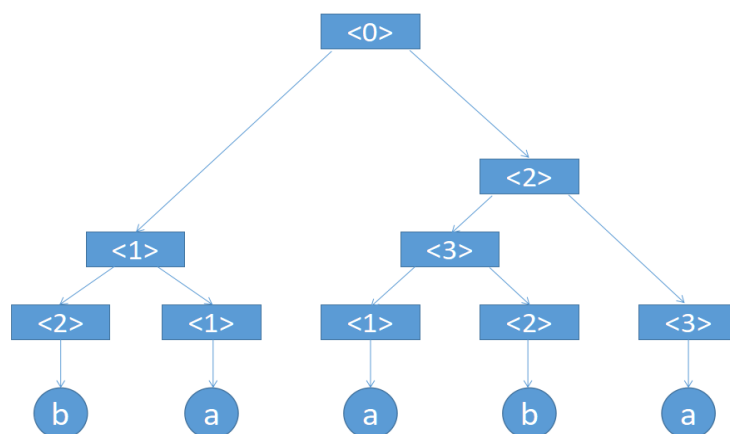
输入样例中的文法和字符串可以构成两棵语法树，如下页图所示。

【要求】

禁止抄袭（包括抄袭往届作业）。

提交截止时间： 5月21日 23:59

同时希望大家完成一份实验报告，实验报告内容包括但不限于：



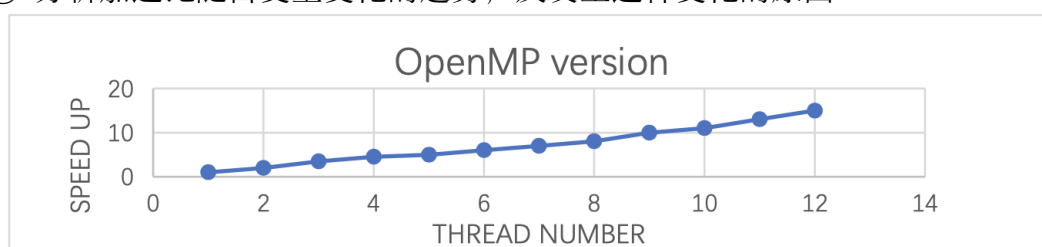
① 如何将任务进行分块的？

② 如何提升运行速度与增强程序扩展性？

③ 如何测量程序运行时间？

④ 请用 Excel 或者其他类似工具作出比较图，比较不同的因素所带来的加速比变化如进程的数量，线程的数量等。写清楚每张图的纵轴横轴是什么，使用了什么工具（如MPI、OpenMP还是Pthreads）。如下图给了一张OpenMP版本的不同线程数的加速比参考样例，选择自己认为比较直观的图表形式即可。

⑤ 分析加速比随自变量变化的趋势，及发生这种变化的原因。



⑥ 最佳的优化方案是什么，该方案的优势是什么？

串行代码

 [serial_CYK.cpp](#)

输入样例

 [input1.txt](#)

 [input2.txt](#)

 [input3.txt](#)